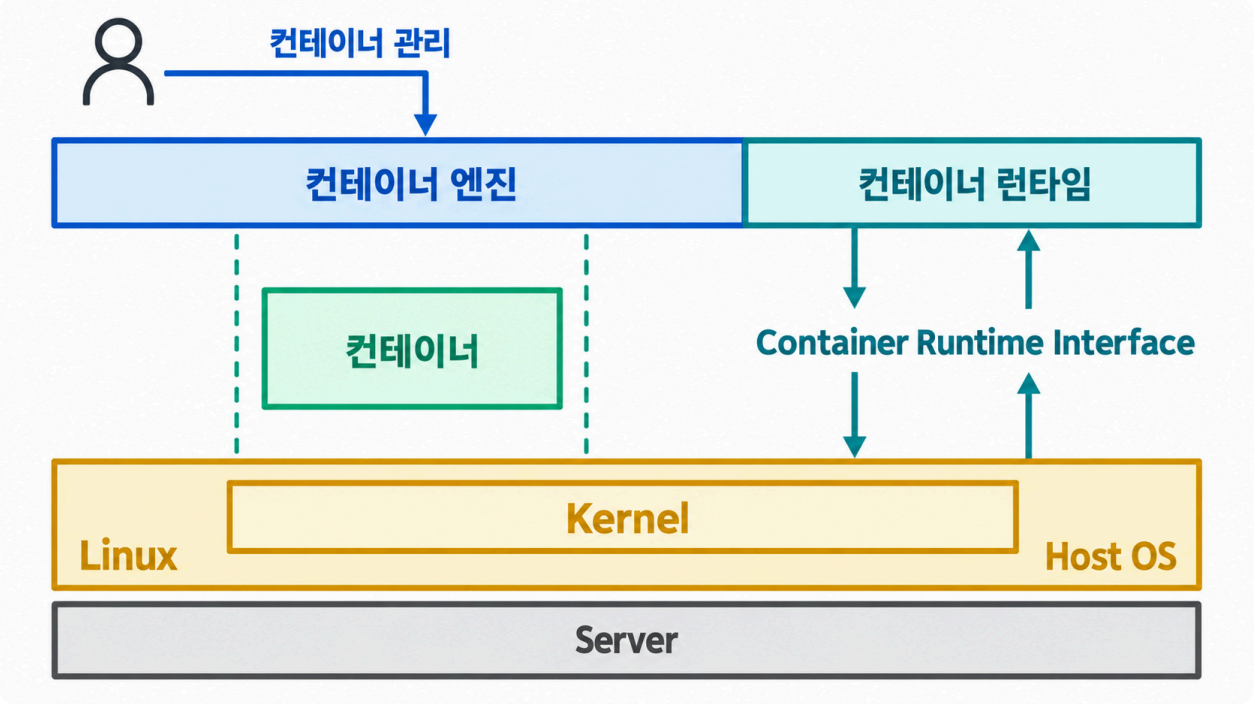
2. Docker

커널의 가상화 기술은 일반 사용자가 직접 다루기에 복잡하다.
이러한 어려움을 해결하기 위해 **Docker** 가 등장했다. Docker는 컨테이너 가상화 기술을 쉽게 사용할 수 있게 해주며 **컨테이너를 간편하게 생성하고 관리할 수 있다.**

도커와 같은 도구를 컨테이너 플랫폼이라고 하며 컨테이너 플랫폼은 컨테이너 엔진과 컨테이너 런타임으로 구성돼 있다.



- **컨테이너 엔진** : 사용자 요청에 따라 컨테이너 관리하며 사용자의 요청을 해석하고 필요한 준비를 마친 후 실행 작업을 런타임에 넘겨준다.

```
사용자 명령 -> 컨테이너 엔진 ------------> 컨테이너 런타임 -> Linux Kernel
                    |-- 이미지가 있는지 확인
                    |-- 없으면 이미지 다운로드
                    |-- 컨테이너 설정 구성
                    |-- 네트워크 준비
```

```
                    |-- 볼륨연결과 리소스 제한 설정
도커에서는 dockerd가 이 관리 역할을 담당한다
```

- **컨테인 런타임** : 커널과 직접 소통해서 실제 컨테이너를 생성하며 요청을 받아 OS의 커널 기능을 사용해서 실제 격리된 프로세스를 만든다.

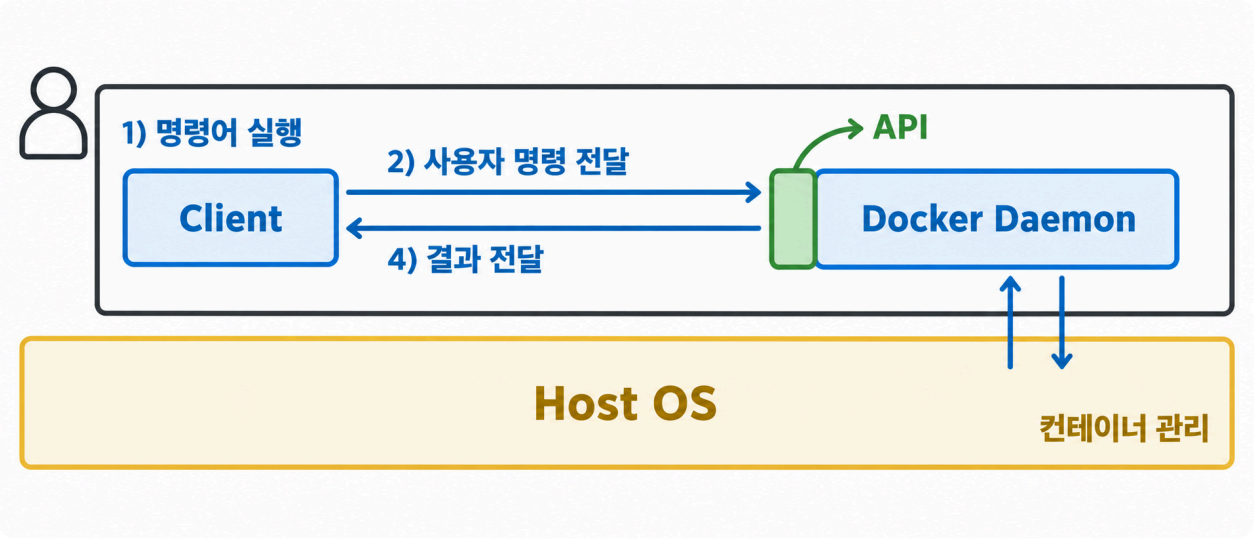
```
namespace 를 이용해 프로세스, 네트워크, 파일 시스템 격리
cgroups 를 이용해 cpu, memory 등의 사용량 제한
컨테이너용 파일 시스템을 구성
컨테이너 내부에서 실행할 프로세스 시작
```

즉, 컨테이너는 특별한 종류의 가상 컴퓨터라기보다 **리눅스 커널이 격리해서 실행하고 있는 하나 이상의 프로세스에 가깝다.**

하이퍼바이저는 격리된 공간을 만들기 위해 하이퍼바이저 소프트웨어가 필요하다면 **컨테이너 가상화**에서 실제 격리를 실행하는 주체는 **커널자체**이다.

2.1 도커의 아키텍처

> 도커는 client - server 모델을 기반으로한다.
>
> Client : 사용자 명령 전달
> Server : 컨테이너 관리하는 도커 데몬 서버로 도커데몬은 dockerd 라고도불린다.

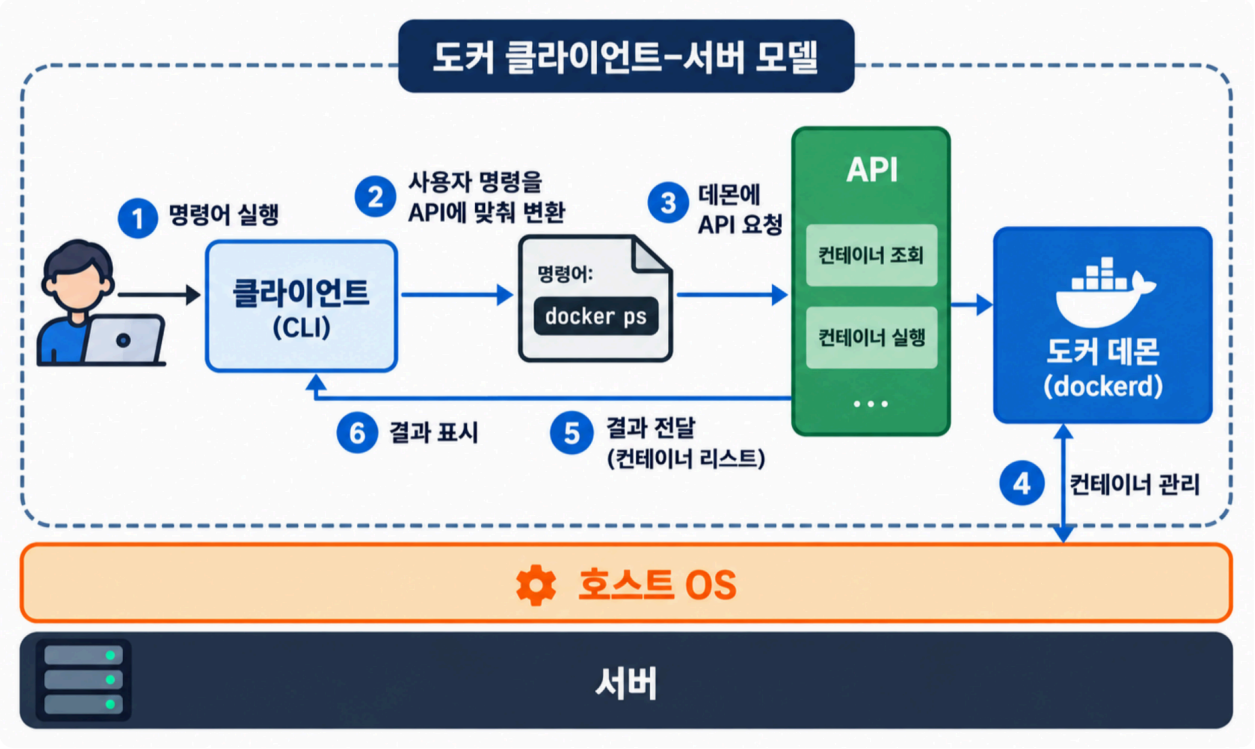


- **명령어 실행** : 클라이언트가 도커 관련 명령어를 실행한다
- **사용자 명령** : 도커 클라이언트가 명령을 도커 데몬에 전달한다
- **컨테이너 관리** : 요청에 따라 컨테이너를 관리한다
- **결과 전달** : 도커 데몬이 클라이언트에게 응답을 제공한다

컨테이너를 생성 및 삭제하려면 도커 데몬에 API 요청을 보내야하지만 복잡하다. 따라서 이를 해결하기 위해 Docker CLI 를 클라이언트로 제공한다.

2.1.1 Docker : Client - Server Model

예제 : 사용자가 컨테이너 목록을 조회하기 위해 docker ps 명령 실행할 때

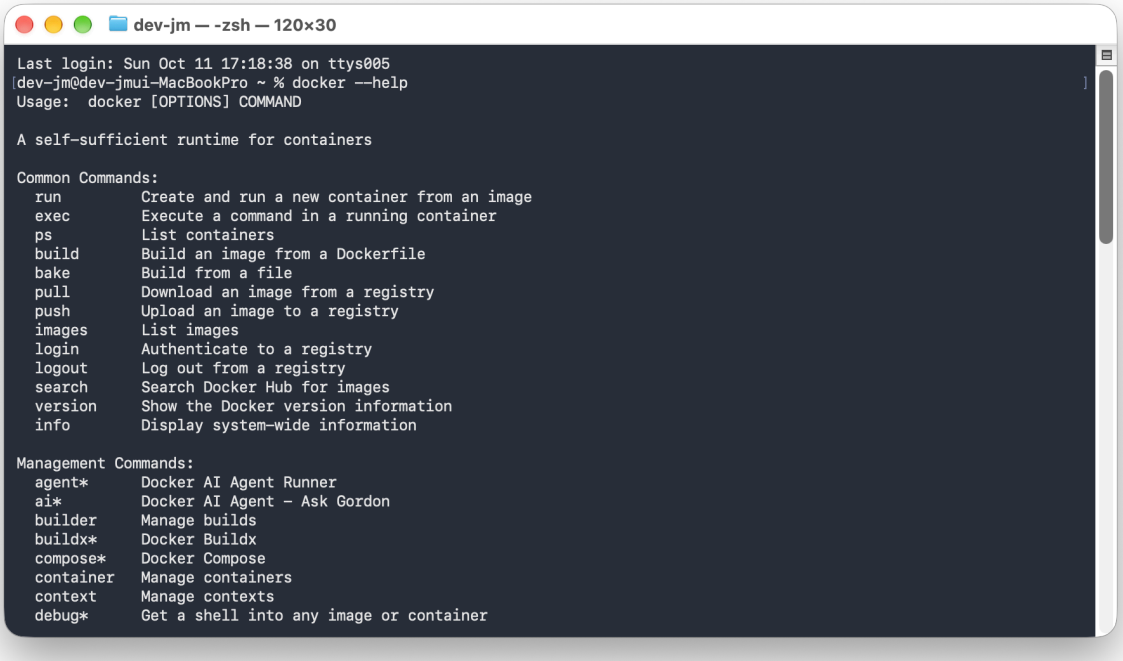


- 클라이언트가 **CLI** 에서 `docker ps` 명령을 실행한다.
- Docker CLI가 명령어를 API 양식에 맞게 변환한다
- Docker CLI 가 도커 데몬에 API 요청을 본내다.
- 도커 데몬이 요청을 분석해 컨테니어 리스트를 조회한다
- 도커 데몬이 조회 결과를 클라이언트에게 응답한다
- CLI는 응답을 사용자가 보기 쉬운 테이블 형태로 화면에 표시한다

2.1.2 컨테이너 실행

도커 명령과 `--help` 옵션을 활용해 도커 명령의 메뉴얼 확인할 수 있다.

```
docker --help
: 메뉴얼 확인
```



예를 들어 `docker image --help` 명령을 하면 docker image 명령어에 관한 설명과 이어서 사용할 수 있는 명령어의 목록을 확인할 수 있다.
도커의 명령어는 **docker 명령, Management Coommand, Command**로 구성되며 관리 커맨드는 보통 생략할 수 있다.

```
docker (Management Command) Command
                대분류              소분류
docker container run .. -> docke run ..
```

`docker --help` 를 실행해 컨테이너가 관련된 명령어를 보면 **Common Commands, Management Commands**라는 명령어들을 확인할 수 있다.
이들은 커맨드를 나타내며 Command Commands 는 커맨드 중 자주 사용되는 명령어를 의미한다. 이 때 **Usage** 부분에서는 컨테이너를 실행할 때 필요한 문법과 옵션을 확인할 수 있다. `[ ]` 로 감싸진 부분은 생략할 수 있음을 나타낸다.

```
Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers
```

```
Common Commands:

  run         Create and run a new container from an
image

  exec        Execute a command in a running container

  ps          List containers

  build       Build an image from a Dockerfile

  bake        Build from a file

  pull        Download an image from a registry

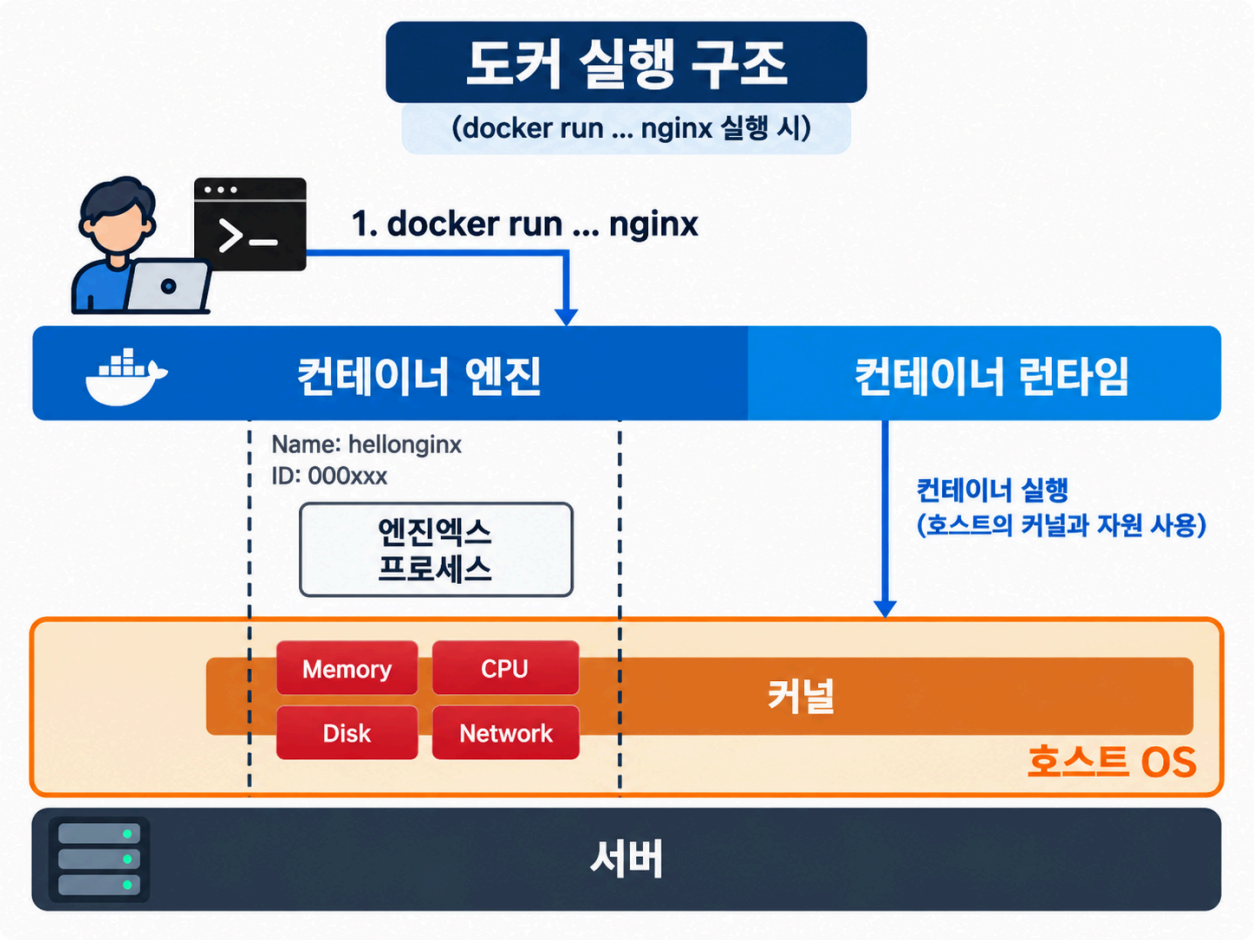
  push        Upload an image to a registry**
```

2.1.2 실행 및 삭제

> 컨테이너를 실행할 때는 `docker run` 명령을 사용하며 `run` 다음에 옵션을 추가한 후 이미지 이름을 지정해야한다.

> 컨테이너를 삭제할 때는 `docker rm` 명령을 사용하며 `rm` 뒤에는 컨테이너의 이름이나 아이디를 입력한다.

```
docker run (option) image-name
docker rm container-name or ID
```



컨테이너 실행 API 에 맞게 변환해 도커 데몬으로 전달한다.

도커 데몬은 요청을 분석한 뒤 컨테이너 런타임을 통해 컨테이너를 생성한다.

이러한 컨테이너는 디스크, 네트워크, 메모리, CPU 등의 자원이 완전히 격리된 공간에 실행되며 여러 서버를 안전하게 운영할 수 있다.